

AI Hackathon - Interactive Support Chatbot

Property	Value
System	Support Chatbot AI
Architectural Style	Multi-Agent Orchestration (Supervisor / Worker)
Framework	Google Agent Development Kit (ADK) & Gemini LLM (Vertex AI)
Status	DRAFT
Document Owner	Support Chatbot Engineering Team
Last Updated	29 July 2026
Space	Engineering / AI Hackathon

This page consolidates the HLD and LLD for the Support Chatbot System. It supersedes the original hackathon draft and reflects the current supervisor/worker agent architecture, DuckDB analytics schema, and ServiceNow ITSM integration.

Table of Contents

- [Executive Summary](#)
 - [1.1 Goals](#)
 - [1.2 Key Capabilities](#)
- [2. High-Level Design \(HLD\) & System Architecture](#)
 - [2.1 Multi-Agent Architecture \(Supervisor / Worker Pattern\)](#)
 - [2.2 Sequence Diagram: Log Diagnostics & Incident Lifecycle](#)
 - [2.2.1 Flow Description](#)
- [3. Project Directory Structure](#)
- [4. Database Design \(DuckDB Diagnostics\)](#)
 - [4.1 Entity-Relationship \(ER\) & View Diagram](#)
 - [4.2 Schema Implementation & Data Dictionary](#)
 - [4.3 Analytics View: v_normalized_api_errors](#)
- [5. ServiceNow REST API Documentation](#)
 - [5.1 Endpoint Overview Matrix](#)
 - [5.2 API Endpoint Specifications](#)
- [6. Environment & Deployment Setup](#)
 - [6.1 Requirements](#)
 - [6.2 Installation Steps](#)
 - [6.3 Environment Configuration \(.env\)](#)
 - [6.4 Local Execution & Server Launch](#)

1. Executive Summary

This document provides the consolidated High-Level Design (HLD) and Low-Level Design (LLD) for the Support Chatbot System. Built using the Google Agent Development Kit (ADK) and powered by Gemini models via Vertex AI, the system uses a multi-agent architecture to

orchestrate user intent, automate local API log analytics through DuckDB with dynamic diagram generation, and manage full-lifecycle IT Service Management (ITSM) workflows through the ServiceNow REST API using an OAuth 2.0 Client Credentials flow.

1.1 Goals

- Provide a single conversational entry point for diagnosing recurring API/integration errors.
- Automate root-cause analysis over locally captured API error logs using natural-language queries.
- Streamline the creation and lifecycle management of ServiceNow incidents directly from chat.
- Establish a modular, extensible agent architecture that can absorb new tools and data sources with minimal rework.

1.2 Key Capabilities

Capability	Description
Conversational Diagnostics	Natural-language querying of the local analytics.duckdb error-log store.
Visual Reporting	Dynamic Markdown / Mermaid chart generation summarizing error trends.
Incident Lifecycle Management	Create, fetch, update ServiceNow incidents.
Multi-Agent Orchestration	Supervisor routes intent to specialized worker agents for separation of concerns.

2. High-Level Design (HLD) & System Architecture

2.1 Multi-Agent Architecture (Supervisor / Worker Pattern)

The system operates under a Supervisor / Worker pattern, dividing responsibilities across three specialized agents:

- **Coordinator Agent (Supervisor):** The primary entry point. Manages initial user interactions, retains conversational context, and routes user intent to the appropriate specialized sub-agent.
- **Data Analyst Agent (Worker):** Queries the local analytical database (analytics.duckdb), aggregates error metrics, and renders Markdown / Mermaid diagram visualizations.
- **ServiceNow Agent (Worker):** Interacts directly with the ServiceNow Incident Management REST API to fetch ticket details, search similar tickets, post work notes, create incidents, and handle binary file uploads.

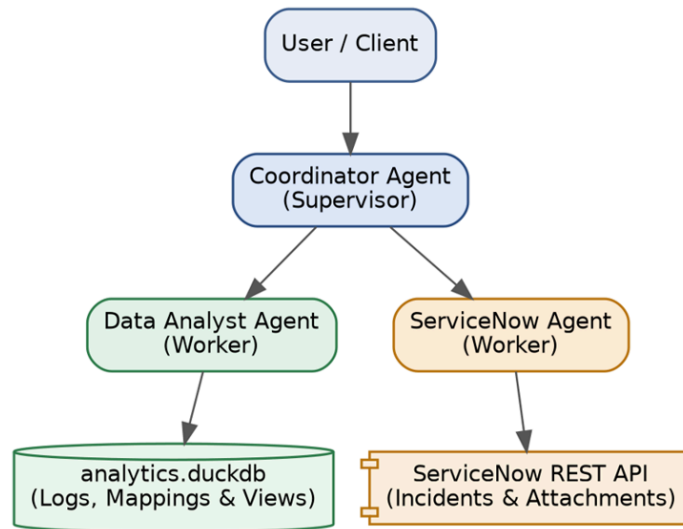


Figure 1 – Supervisor / Worker agent architecture

Architecture note: the Coordinator never talks to analytics.duckdb or the ServiceNow API directly — all external I/O is delegated to worker agents, keeping the supervisor stateless with respect to data access and easy to reason about.

2.2 Sequence Diagram: Log Diagnostics & Incident Lifecycle

The sequence below traces a full end-to-end interaction: a user requests a log diagnosis, receives an error summary with a visualization, and then asks the system to open a ServiceNow incident from the findings.

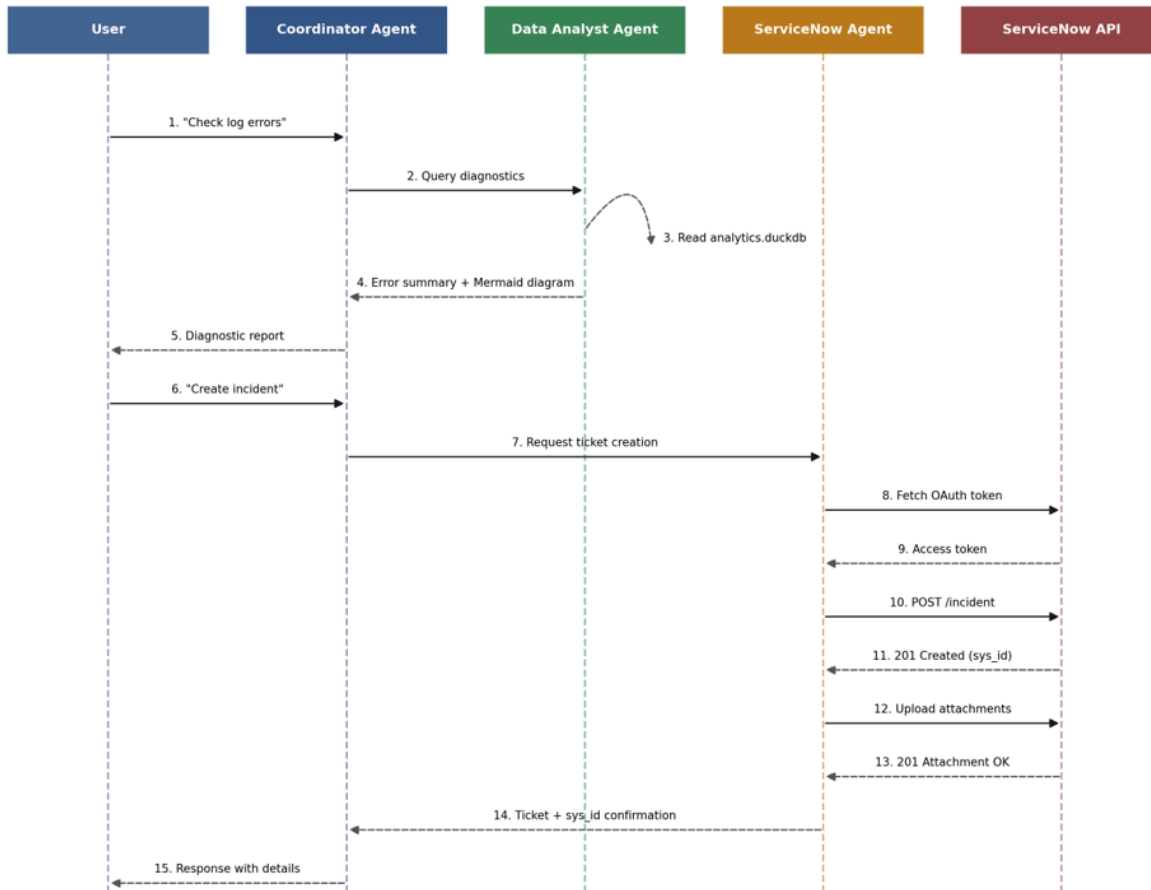


Figure 2 – End-to-end sequence: diagnostics request through incident creation

2.2.1 Flow Description

1. User asks the Coordinator Agent to check for log errors.
2. Coordinator delegates the diagnostic query to the Data Analyst Agent.
3. Data Analyst Agent reads and aggregates records from analytics.duckdb.
4. An error summary plus a generated Mermaid diagram is returned to the Coordinator, then relayed to the user.
5. User requests that an incident be created from the diagnosis.
6. Coordinator hands off the request to the ServiceNow Agent.
7. ServiceNow Agent obtains an OAuth 2.0 bearer token, creates the incident, and returns the ticket number and sys_id.
8. Coordinator relays the confirmation and ticket details back to the user.

3. Project Directory Structure

The repository follows a standard Python package layout, keeping agent orchestration, database access, and tool definitions cleanly separated:

```

support_chatbot/
├── src/
│   ├── support_chatbot/
│   │   ├── __init__.py
│   │   └── agent/
│   │       ├── __init__.py

```

```

| | └─ agent.py      # Multi-agent orchestration
| | |               # (Coordinator, Data Analyst, ServiceNow)
| | └─ prompts.py    # System instructions & output formatting
rules
| └─ config.py       # Centralized Pydantic configuration (from
.env)
| └─ database/
| | └─ __init__.py
| | └─ connection.py # DuckDB connection manager (read-only
enforcement)
| | └─ sql/
| | | └─ api_error_logs.sql
| | | └─ country_mappings.sql
| | | └─ v_normalized_api_errors.sql
| └─ tools/
| | └─ __init__.py
| | └─ db_tools.py   # LLM-callable functions for DuckDB
analysis
| | └─ servicenow_tools.py # LLM-callable functions for
ServiceNow ITSM ops
└─ .adk/             # Local ADK session and cache store
└─ .env              # Environment credentials and variables
└─ .gitignore
└─ .idea/            # PyCharm / IntelliJ configuration files
└─ data/             # Holds local .duckdb database files
└─ pyproject.toml     # Build system metadata (supports pip install
-e .)
└─ README.md         # System documentation
└─ requirements.txt   # Project dependencies
└─ agent.py          # Primary entry point for ADK CLI

```

4. Database Design (DuckDB Diagnostics)

Local diagnostic analytics are managed using an embedded DuckDB database (analytics.duckdb) that captures raw API log data, resolves country codes, and projects normalized error classifications for downstream reporting.

4.1 Entity-Relationship (ER) & View Diagram

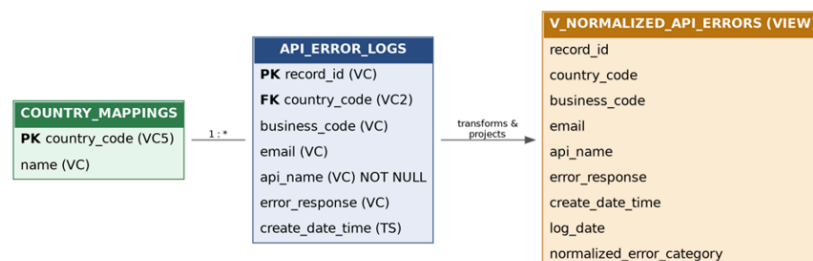


Figure 3 – Entity-relationship diagram and analytics view lineage

4.2 Schema Implementation & Data Dictionary

Table 1: country_mappings

src/support_chatbot/database/sql/country_mappings.sql

Stores geographical reference mapping for region and country resolution.

```
CREATE TABLE IF NOT EXISTS country_mappings (  
    country_code VARCHAR(5) PRIMARY KEY,  
    name VARCHAR  
);
```

Table 2: api_error_logs

src/support_chatbot/database/sql/api_error_logs.sql

Stores raw incoming error logs and API transaction failure records.

```
CREATE TABLE IF NOT EXISTS api_error_logs (  
    record_id    VARCHAR PRIMARY KEY,  
    country_code VARCHAR(2),  
    business_code VARCHAR,  
    email        VARCHAR,  
    api_name     VARCHAR NOT NULL,  
    error_response VARCHAR,  
    create_date_time TIMESTAMP,  
    FOREIGN KEY (country_code) REFERENCES  
    country_mappings(country_code)  
);
```

Column Name	Data Type	Constraints	Description
record_id	VARCHAR	PRIMARY KEY	Unique execution record identifier
country_code	VARCHAR(2)	FK	ISO country identifier
business_code	VARCHAR	Nullable	Organization unit or business domain code
email	VARCHAR	Nullable	User email address associated with the transaction
api_name	VARCHAR	NOT NULL	API endpoint or service

			invocation target
error_response	VARCHAR	Nullable	Raw trace, exception details, or response body
create_date_time	TIMESTAMP	Nullable	Exact log timestamp

4.3 Analytics View: v_normalized_api_errors

src/support_chatbot/database/sql/v_normalized_api_errors.sql

Projects structured error categorizations from raw unstructured trace messages to enable automated diagnostics, categorization, and reporting via the Data Analyst Agent. The view applies a cascading CASE expression to classify api_error_logs.error_response into a normalized_error_category, grouped conceptually as follows:

- **Service Point Creation errors:** Missing building / floor / department references in PNOL.
- **Dynamic Visit Processing errors (code 20093):** Unable to find LOCATION, TASK, RECOMMENDATION_REF, PREP, INFESTATION_LEVEL, SITE_REF, or EMPLOYEE values across their respective source tables.
- **Dynamic Division & Record errors (code 20097 / 20010 / 20015):** Failures saving site division data (missing building, floor, or site) or record creation/lookup failures in iCABS.
- **Standard fixed error codes:** A defined set of codes (20091, 20087, 20081, 20012, 20011, 20008, 20004, 20003, 20086) each mapped to a human-readable category.
- **General fallback rules:** Any remaining 20093-prefixed or generic API-call failures default to a catch-all category (e.g. 50000: Internal Server Error); anything unmatched passes through as the raw error_response.

```
CREATE OR REPLACE VIEW v_normalized_api_errors AS
SELECT
  record_id,
  country_code,
  business_code,
  email,
  api_name,
  error_response,
  create_date_time,
  CAST(create_date_time AS DATE) AS log_date,
  CASE
    -- Service Point Creation
    WHEN TRIM(error_response) ILIKE '%1:%'
      THEN 'Service point creation error due to missing building /
        floor / department in PNOL'

    -- Dynamic Visit Processing (abridged — full case list in repo)
    WHEN TRIM(error_response) ILIKE '%20093:% Unable to find LOCATION%'
      THEN '20093: Unable to find LOCATION with value'
```

```

WHEN TRIM(error_response) ILIKE '%20093:%Unable to find TASK%'
    THEN '20093: Unable to find TASK with value in TASK_ACTION'

-- Dynamic Division & Record Errors
WHEN TRIM(error_response) ILIKE '%20097:%Missing building%'
    THEN '20097: Error while saving site division data: Missing building'
WHEN TRIM(error_response) ILIKE '%20010:%Record Not Found%'
    THEN '20010: Record Not Found in iCABS'

-- Standard Fixed Error Codes
WHEN TRIM(error_response) ILIKE '%20091:%WorkPoint Must Be Supplied%'
    THEN '20091: WorkPoint Must Be Supplied'
WHEN TRIM(error_response) ILIKE '%20081:%This WorkOrder Has Been Closed%'
    THEN '20081: WorkOrder Closed'

-- General Fallback Rules
WHEN TRIM(error_response) ILIKE '%20093:%' THEN '20093: Visit Processing Error'
WHEN TRIM(error_response) ILIKE '%Error while calling API %'
    THEN '50000: Internal Server Error'

ELSE error_response
END AS normalized_error_category
FROM api_error_logs;

```

5. ServiceNow REST API Documentation

5.1 Endpoint Overview Matrix

Operation	Endpoint	HTTP Method	Auth Type	
OAuth Token	/oauth_token.do	POST	Client Credentials	
Create Incident	/api/now/table/incident	POST	Bearer Token	
Get Incident	/api/now/table/incident/{sys_id}	GET	Bearer Token	
Update Incident	/api/now/table/incident/{sys_id}	PATCH	Bearer Token	

5.2 API Endpoint Specifications

1. OAuth 2.0 Token Generation (Client Credentials Flow)

Obtains a bearer access token for authenticating subsequent API requests.

- **Endpoint:** /oauth_token.do
- **HTTP Method:** POST

- **Content-Type:** application/x-www-form-urlencoded

Form Parameters:

- **grant_type** (string, required) — value must be client_credentials (or password).
- **client_id** (string, required) — ServiceNow application client ID.
- **client_secret** (string, required) — ServiceNow application client secret.

Success Response (200 OK):

```
{
  "access_token": "eyJhbGciOi...",
  "token_type": "Bearer",
  "expires_in": 1800,
  "refresh_token": "rF83a21..."
}
```

2. Create Incident

Creates a new IT Service Management ticket record in ServiceNow.

- **Endpoint:** /api/now/table/incident
- **HTTP Method:** POST

Headers:

- **Authorization:** Bearer <access_token>
- **Content-Type:** application/json
- **Accept:** application/json

Query Parameters:

- **sysparm_input_display_value** (boolean, optional) — set to true to resolve display strings for reference fields.

Request Body:

```
{
  "short_description": "API Failure: Image Service Timed Out",
  "description": "Error observed in logs during transaction check.",
  "caller_id": "john.doe@example.com",
  "urgency": "2",
  "impact": "2"
}
```

Success Response (201 Created):

```
{
  "result": {
    "sys_id": "a9c381ef972301103b...",
    "number": "INC0010045",
    "state": "1",
    "urgency": "2",
    "impact": "2",
    "short_description": "API Failure: Image Service Timed Out"
  }
}
```

```
}  
}
```

3. Retrieve Incident Details

Fetches current status, assigned group, and field values for a specific ticket.

- **Endpoint:** /api/now/table/incident/{sys_id}
- **HTTP Method:** GET

Headers:

- **Authorization:** Bearer <access_token>
- **Accept:** application/json

Path Parameters:

- **sys_id** (string, required) — unique 32-character record identifier.

Success Response (200 OK):

```
{  
  "result": {  
    "sys_id": "a9c381ef972301103b...",  
    "number": "INC0010045",  
    "incident_state": "2",  
    "assigned_to": "IT Support Group",  
    "comments": "Investigating log traces."  
  }  
}
```

4. Update Incident

Updates work notes, incident state, or priority fields on an existing record.

- **Endpoint:** /api/now/table/incident/{sys_id}
- **HTTP Method:** PATCH

Headers:

- **Authorization:** Bearer <access_token>
- **Content-Type:** application/json
- **Accept:** application/json

Request Body:

```
{  
  "work_notes": "Added diagnostic error stack trace from analytics.duckdb  
log extraction.",  
  "state": "2"  
}
```

Success Response (200 OK):

```
{  
  "result": {  
    "sys_id": "a9c381ef972301103b...",
```

```
"number": "INC0010045",  
"state": "2",  
"sys_updated_on": "2026-07-29 12:00:00"  
}  
}
```

6. Environment & Deployment Setup

6.1 Requirements

- Python 3.10+
- **Google ADK CLI** (pip install google-adk)
- Google Cloud Project with Vertex AI enabled

6.2 Installation Steps

```
# Clone repository and enter root directory  
cd support_chatbot  
  
# Install project dependencies and local editable package  
pip install -r requirements.txt  
pip install -e .
```

6.3 Environment Configuration (.env)

Create a .env file in the project root containing the following variables:

```
# Google Cloud / Vertex AI Configuration  
GOOGLE_GENAI_USE_VERTEXAI=1  
GOOGLE_CLOUD_PROJECT="your-gcp-project-id"  
GOOGLE_CLOUD_LOCATION="us"  
  
# GOOGLE_APPLICATION_CREDENTIALS="/path/to/key.json" # Optional  
# if authenticated via gcloud CLI  
  
# ServiceNow API Credentials (OAuth 2.0 Client Credentials Flow)  
SERVICENOW_INSTANCE_URL="https://yourinstance.service-now.com"  
SERVICENOW_CLIENT_ID="your_oauth_client_id"  
SERVICENOW_CLIENT_SECRET="your_oauth_client_secret"  
# SERVICENOW_USERNAME="optional_system_user_for_caller_id"  
# SERVICENOW_PASSWORD="optional_system_user_password"
```

6.4 Local Execution & Server Launch

```
# 1. Clear ADK cache (crucial after configuration or editable installation
changes)

rm -rf .adk

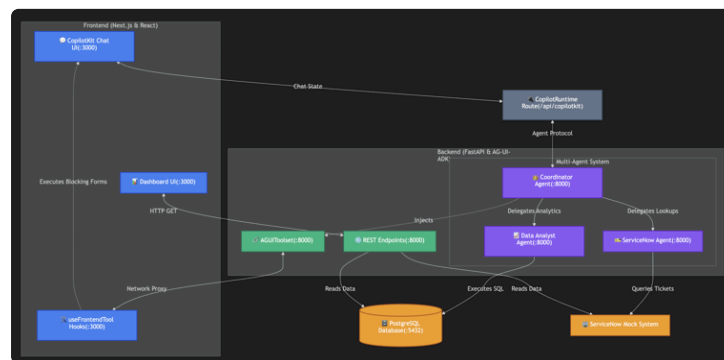
# 2. Run the ADK development server pointing explicitly to the agent
module

adk run support_chatbot.agent.agent
```

6.5 Frontend Dashboard Architecture

The Frontend project is built on a decoupled, agentic architecture that seamlessly integrates a modern React dashboard with a powerful, multi-agent Python backend via CopilotKit.

6.5.1 High-Level Architecture Diagram



6.5.2 Component Breakdown

1. Frontend (Next.js)

- **Dashboard UI (page.tsx):** Displays the API Error Logs and ServiceNow Incidents. Uses Recharts for the dynamic Pie Chart and Tailwind v4 for class-based dark/light mode toggling.
- **CopilotKit Client (@copilotkit/react-core):** Powers the chat interface and exposes interactive frontend tools (like the popup ticket creation modal) using the useFrontendTool hook.

2. CopilotRuntime Route

- Serves as the middleware bridging the Next.js React frontend to the Python agent backend. It streams LLM tokens and coordinates tool call network requests.

3. Backend (FastAPI + Google ADK)

- **Coordinator Agent:** The root agent that acts as a router. It holds the AGUIToolset, which dynamically syncs your Next.js frontend tools into its LLM context.
- **Data Analyst Agent:** A specialized sub-agent strictly restricted to querying the PostgreSQL database to analyze API logs.
- **ServiceNow Agent:** A specialized sub-agent for fetching existing ticket details and searching the mock ITSM system.
- **REST Endpoints:** Standard FastAPI routes that serve the raw data arrays to the React dashboard.